

Oktav Hatası: Kapsamlı Araştırma Raporu

YIN tabanlı perde tespitinde oktav / alt-harmonik hataları — akademik literatür, bireysel çalışmalar ve mühendislik pratikleri

KlariVision projesi için hazırlanmıştır · 4 Eylül 2026 · Giriş seviyesi sinyal işleme bilgisiyle okunabilecek şekilde yazılmıştır

İçindekiler

1. Kısa cevap: çözümlerin sıralaması
2. Temel: oktav hatası neden kaçınılmaz bir tuzak?
3. YIN'in kendi içindeki savunma hatları (ölçülmüş katkılarıyla)
4. Çözümler — en iyi sonuç verenden başlayarak
5. Tuzaklar — yapılmaması gerekenler
6. KlariVision için somut uygulama sırası
7. Kaynakça

1. Kısa cevap: çözümlerin sıralaması

#	Çözüm	Ne kadar iyi	Maliyet	Bu proje için uygunluk
1	pYIN (aday dağılımı + HMM/Viterbi)	Oktav hatası %0,5–1,7; YIN'e göre recall 0,95 → 0,98	Düşük (YIN'in ~1,5 katı)	★★★★★ Çevrimdışı analizde bugün uygulanabilir
2	Aday + dinamik programlama izleme (RAPT / Praat tarzı; gerçek zamanda <i>fixed-lag</i> Viterbi)	Gross error %0,65 (RAPT*) — klasik yöntemlerin en iyisi	Düşük–orta	★★★★★ FixedLagPitchTracker kodda zaten mevcut
3	Spektral harmonik kanıtla veto/puanlama (SWIPE', TWM, SHS/HPS)	Alt-harmonik hatalarını kökten keser; klarnet için özellikle uygun	Orta (FFT gerekir)	★★★★★ swipe_prime + harmonic_arbitration mevcut; eksik olan bağlantı
4	Sinir ağı tabanlı tahminçiler (SwiftF0, RMVPE, CREPE)	Genel doğruluk %90,2 / %87,2 / %85,3; gürültüde fark açılıyor	SwiftF0 ≈ 96 bin parametre; CPU'da CREPE'ten 40–90x hızlı	★★★☆☆ iOS'ta mümkün; eğitim verisi ve klarnet uyumu risk
5	Bayesçi harmonik model (model derecesi seçimiyle)	Oktav hatasını yapısal olarak çözer	Yüksek	★★☆☆☆ Akademik olarak en zarif; gerçek zamanda ağır
6	Ucuz mühendislik önlemleri (arama aralığı, pencere boyu, Adım 6, medyan filtre, histerezis)	YIN'in kendi ölçümündeki %10 → %0,5 düşüşün büyük kısmı buradan gelir	Neredeyse sıfır	★★★★★ Önce bunlar yapılmalı — mevcut YIN'de eksikler var

2. Temel: oktav hatası neden kaçınılmaz bir tuzak?

2.1 Periyodikliğin matematiksel kusuru

Bir ses “periyodik” ise, T periyoduyla kendini tekrar eder:

$$x(t) = x(t + T)$$

Ama dikkat: eğer bu doğruysa, **şu da otomatik olarak doğrudur**:

$$x(t) = x(t + 2T) = x(t + 3T) = x(t + 4T) = \dots$$

Yani sinyal 2T ile de, 3T ile de “periyodiktir”. Matematiksel olarak periyot **tek bir sayı değil, bir kümedir**: {T, 2T, 3T, ...}. Biz bu kümenin *en küçük* elemanını istiyoruz. de Cheveigné ve Kawahara'nın YIN makalesindeki tanım tam olarak budur: periyot, kümenin en küçük pozitif elemanıdır.

Oktav hatası, algoritmanın bu kümeden **yanlış elemanı seçmesidir**:

- **“Çok alçak” hata** (sub-harmonic / pitch halving): T yerine 2T seçilir → frekans yarıya iner → bir oktav pes duyulur.
- **“Çok yüksek” hata** (pitch doubling): T yerine T/2 seçilir → bir oktav tiz.

YIN makalesi burada önemli bir uyarı yapar: buna “oktav hatası” demek aslında yanlıştır, çünkü hata her zaman 2'nin kuvveti oranında olmaz — 3T seçilirse bir *duodesim* (on ikili) kayarsınız. **Bu ayrım bu proje için hayatidir** (bkz. bölüm 2.3).

2.2 Otokorelasyon ve fark fonksiyonu — grafikte ne görüyoruz?

Otokorelasyon (ACF). Sinyali kendisiyle τ kadar kaydırıp çarpıp toplarsınız. Sinyal kendisiyle “hizalanınca” sonuç büyük çıkar:

$$r(\tau) = \sum x(j) \cdot x(j+\tau)$$

Zirveler $\tau = 0, T, 2T, 3T...$ noktalarında oluşur. Sorun şudur: zirveler *aynı yükseklikte* olmadığı için algoritma yanlış zirveyi seçebilir. Sinyalin genliği zamanla büyüyorsa (crescendo, atak), ileri lag'lerdeki zirveler daha yüksek çıkar ve algoritma **sistemik olarak “çok alçak” hata yapar**.

Fark fonksiyonu (YIN'in 2. adımı). Çarpmak yerine çıkarıp karesini alırsınız:

$$d(\tau) = \sum (x(j) - x(j+\tau))^2$$

Şimdi doğru periyotta değer **sıfıra iner** (çukur), yükselmez. Bu, genlik değişimine karşı bağışıklık kazandırır. YIN makalesinin kendi ölçümünde bu tek değişiklik hata oranını **%10,0'dan %1,95'e** düşürmüştür.

2.3 Klarnet neden özel bir kâbus?

Klarnet **silindirik, bir ucu kapalı** bir borudur. Bunun akustik sonuçları:

- Sadece **tek harmonikler** güçlüdür: $f, 3f, 5f, 7f...$
- **2. harmonik (2f) pratikte yoktur.**
- Chalumeau bölgesinde **temel frekans, kendi 3. harmoniğinden kat kat zayıf** olabilir.
- Klarnet oktavdan (2x) değil, **on ikiliden (3x) aşırı üfler**.

Bunun oktav hatası açısından anlamı şudur: otokorelasyon/YIN ailesinin fark fonksiyonu, spektrumda **hiç enerji olmayan** bir frekansta bile derin bir çukur üretebilir. Gerçek nota $3f$ ise, f konumunda spektral olarak hiçbir şey yokken $d(\tau)$ orada da çukur yapar. Projenin kendi kodundaki yorumlarda buna çok isabetli bir isim verilmiş: **hayalet periyot (ghost subharmonic)**.

Sonuç: Bu projede karşılaşılan hata, tipik “insan sesi oktav hatası” değil, çoğunlukla **$f/3$ tipi bir alt-harmonik hatasıdır**. Literatürdeki standart “2x kontrol et” reçeteleri burada **yetersiz kalır**: 3x de kontrol edilmeli, ama 2x kısıtı **gevşetilmelidir**, çünkü klarnette 2f'nin yokluğu normaldir, hata kanıtı değildir.

Koddaki durum: `core/include/klarivision/core/hapt.hpp` içindeki “çift harmonikler tamamen kısıtsız bırakılırken yarım ve üçte-bir periyot kanıtına karşı puanlama” tasarımı, literatürdeki genel reçetelerden daha doğru bir yaklaşımdır. Korunmalıdır.

3. YIN'in kendi içindeki savunma hatları

YIN makalesinin en değerli kısmı, her adımın hata oranına katkısını *ölçmesidir*. Aşağıdaki tablo, küçük bir konuşma veritabanında kaba hata oranını (gross error rate) adım adım gösterir:

Adım	Ne yapar	Hata oranı	Kazanç
1	Ham otokorelasyon — zirve ara	%10,00	—
2	Fark fonksiyonu $d(\tau)$ — genlik duyarlılığını yok eder	%1,95	–8,05 puan
3	Kümülatif ortalama normalizasyon $d'(\tau)$ — “çok yüksek” hataları keser	%1,69	–0,26 puan
4	Mutlak eşik (0,1) — “çok alçak” (alt-harmonik) hataları keser	%0,78	–0,91 puan
5	Parabolik interpolasyon — ince frekans doğruluğu	%0,77	–0,01 puan
6	Best local estimate — komşu zaman noktalarında daha iyi tahmin arar	%0,50	–0,27 puan

(a) 3. adım — kümülatif ortalama normalizasyon

$$d'(\tau) = d(\tau) / [(1/\tau) \cdot \sum_{j=1..T} d(j)]$$

Her değeri, kendisinden kısa lag'lerdeki ortalamaya böler. Fonksiyon 0 yerine 1'den başlar ve ancak $d(\tau)$ ortalamanın altına düştüğünde 1'in altına iner. Böylece $\tau=0$ çukuru ve formant kaynaklı sahte kısa-lag çukurları elenir. Ek fayda: arama aralığına üst frekans sınırı koyma zorunluluğu ortadan kalkar.

(b) 4. adım — mutlak eşik

En küçük τ 'yu, d' değeri eşik altında kalan yerel minimum olarak seçmek. Buradaki “**en küçük**” kelimesi kritiktir: periyot kümesinin en küçük elemanını zorlar, yani $2T/3T$ 'ye kaymayı engeller.

Eşik fiziksel anlamı zariftir: $d'(T)$, **aperiyodik güç / toplam güç** oranına orantılıdır. Eşik 0,15 seçmek, “periyodik” saymak için sinyalin en fazla %15'inin aperiodyik olmasına izin veriyorum demektir.

(c) 6. adım — best local estimate

Fikir şudur: bir tahmin, periyodun belirli bir *fazında* bozulur, ama komşu bir zaman noktasında aynı nota için d' değeri çok daha düşüktür. Algoritma her zaman noktası için $[t - T_{\max}/2, t + T_{\max}/2]$ aralığında **en düşük $d'(T)$ veren komşuyu** bulur, onu ilk tahmin kabul eder ve arama aralığını o tahminin $\pm 20\%$ siyle sınırlayarak algoritmayı tekrar çalıştırır ($T_{\max} = 25$ ms).

Bu tek adım hata oranını **%0,77'den %0,50'ye** düşürüyor — yani kalan hataların **üçte birini** temizliyor. Makale, bunun medyan filtreden veya dinamik programlamadan farkını da vurgular: karar *sürekliliğe* değil, **kaliteye** dayanır.

Koddaki durum — doğrudan bulgu: `src/klarivision/pitch/yin.py` içinde Adım 1–5 mevcut, **Adım 6 yok**. Ayrıca `minimum_frequency_hz = 110.0` ile Bb klarnetin en pes yazılı Mi'si (duyulan $\approx 146,8$ Hz) arasında bir oktav-altı hata için bol yer bırakılmış. Bu iki nokta, ileri teori gerektirmeden ölçülebilir kazanç verir.

4. Çözümler — en iyi sonuç verenden başlayarak

Çözüm 1 — pYIN: “tek tahmin verme, aday listesi ver, sonra en tutarlı yolu seç”

Mauch & Dixon, ICASSP 2014, Queen Mary University of London.

Problemin teşhisi

Klasik YIN her kare için **tek bir** tahmin üretir. Sonradan uygulanan her düzeltme (medyan filtre, atlama engelleme) bu tek tahmini yumuşatmaya çalışır — ama **atılmış olan doğru cevabı geri getiremez**. Bir karede YIN f/3'ü seçtiyse, doğru f değeri artık hiçbir yerde yoktur. Makalenin ifadesiyle: bir kez hata yapıldıysa gerçek değer kurtarılamaz.

Aşama 1 — Eşiği bir sayı olmaktan çıkarıp dağılıma çevirmek

YIN'de $\text{threshold} = 0.15$ gibi tek bir sayı seçersiniz. pYIN şunu fark eder: **çoğu hatalı karede, doğru cevabı verecek bir eşik değeri vardır** — sadece siz onu önceden bilemezsiniz. O hâlde hepsini birden deneyin:

- Eşik değerleri: 0,01'den 1,00'a, 0,01 adımlarla → **N = 100 eşik**
- Her eşik için **ön olasılık** atanır: ortalaması 0,10 / 0,15 / 0,20 olan **Beta dağılımları**
- Her eşik için YIN çalıştırılır — bu tek bir YIN döngüsünde yapılabilir, ek maliyet çok düşüktür
- Bir τ değeri kaç eşik altında kazanıyorsa, o eşiklerin olasılıkları toplanır → **P(τ = periyot)**
- Eşik altında hiçbir şey bulunamadığında kullanılan “mutlak minimum” stratejisine küçük bir olasılık verilir: **$p_a = 0,01$**
- Toplam olasılık 1'e tamamlanmazsa, **artan kütle “sessiz/ötümsüz” olasılığı** olarak yorumlanır — voicing kararı bedavaya gelir

Çıktı tek bir frekans değil, **{(f₁, p₁), (f₂, p₂), ...}** aday-olasılık çiftleridir. Kritik özellik: klasik YIN'in cevabı **her zaman bu listenin içindedir**, dolayısıyla pYIN hiçbir zaman YIN'den kötü olamaz.

Makaledeki recall tablosu bu argümanı doğrular:

Yöntem	Orijinal kayıt	Canlı kayıt	Telefon kaydı
Tüm aday kümesi	0,993	0,750	0,953
YIN (0,10)	0,988	0,648	0,880
YIN (0,15)	0,989	0,644	0,843
YIN (0,20)	0,980	0,632	0,803

Zor koşullarda doğru cevap %75 oranında **aday listesinde vardır**, ama tek eşikli YIN onu ancak %64 oranında seçebilmektedir. Aradaki 11 puan saf kayıptır.

Aşama 2 — HMM + Viterbi ile en tutarlı yolu seçmek

Giriş seviyesi açıklama: Elinizde her zaman karesi için birkaç aday var. Bunları bir izgara gibi düşünün — yatay eksen zaman, dikey eksen frekans; her karede birkaç nokta yanıyor. Viterbi algoritması bu ızgarada hem **parlak noktalardan geçen** hem de **zıplamayan** en iyi yolu bulur. “En iyi”nin tanımı iki puanın çarpımıdır: (adayın kendi olasılığı) × (önceki kareden buraya geçme olasılığı).

pYIN'in somut kurulumu:

- Durum uzayı:** 55 Hz (A1) – 880 Hz (A5) arası 4 oktav, **10 sent** (0,1 yarım ton) adımlarla **M = 480 kutu**
- Her frekans kutusunun **ötümlü ve ötümsüz** iki hâli → toplam 2M durum
- Gözlem olasılığı:** Aşama 1'in aday olasılıkları doğrudan kullanılır (seyrek vektör)
- Voicing geçişi:** aynı kalma 0,99 — değişme 0,01
- Perde geçişi:** **üçgen ağırlıklı** dağılım; en fazla **25 kutu = 2,5 yarım ton / kare** sıçramaya izin verir, tepe 0 sentte
- Geçiş matrisinin seyrekliğinden faydalanan verimli bir Viterbi ile çözülür
- Parametreler: kare 2048 örnek (46,4 ms), hop 256 örnek (5,8 ms)

Oktav hatasına etkisi — anahtar cümle: Perde geçiş penceresi kare başına 2,5 yarım tonla sınırlıdır. **Bir oktav 12 yarım tondur**. Yani tek bir karede oktav sıçraması yapmak modelde neredeyse **imkânsızdır**. Oktav hatası ancak birden fazla kare boyunca ısrarla daha iyi kanıt sunarsa kazanabilir — ki bu da gerçek bir nota değişimi demektir.

Ölçülen sonuçlar

RWC veritabanından sentezlenmiş 30+ saat şarkı, Audio Degradation Toolbox ile beş bozulma varyantı:

- Oktav hatası: %0,5 / %0,9 / %1,7** (sırasıyla Beta ortalaması 0,10 / 0,15 / 0,20)

- Recall medyanları: **0,977 / 0,982 / 0,984** — teorik üst sınır olan 0,98'e neredeyse değiyor
- Klasik YIN'in recall medyanları: hepsi **0,951'in altında**
- Precision: pYIN 0,984–0,987; klasik YIN 0,970–0,985
- Voicing recall %92,5–95,0; specificity %88,9–91,9

Makaledeki nitel örnek bu projenin durumuyla birebir örtüşür: “Happy Birthday”ın son dört notasında YIN **çok sayıda oktav hatası** yaparken pYIN temiz izi çıkarır; yazarlar sebebi şarkıcının **alışılmadık nefesli tınısı** olarak gösterir. Nefesli tını = zayıf temel + güçlü üst harmonikler = **klarnetin chalumeau bölgesiyle aynı fiziksel durum**.

Kritik ayırım — sadece yumuşatma yetmiyor

Makale kontrol grubu olarak “YIN + Smoothing” (YIN+S) de çalıştırmıştır: tek eşikli YIN'in çıktısı aynı HMM'den geçirilir.

Yöntem	Recall	Precision	F-ölçütü
YIN 0,15	0,935	0,981	0,981
YIN+S 0,15	0,918	0,986	0,894
pYIN 0,15	0,982	0,986	0,960

YIN+S'in recall'ı **orijinal YIN'den daha kötüdür**. Sebep: yumuşatılacak tek bir tahmin vardır, dolayısıyla recall YIN'in performansı ile tavanlanır. **Ders: Viterbi'yi tek adaylı bir çıktıya uygulamak sizi kurtarmaz. Önce çoklu aday üretmelisiniz.**

Koddaki durum: `src/klarivision/pitch/pyin.py` zaten librosa üzerinden pYIN'i sarmalıyor (5 sent çözünürlük, 110–2000 Hz). Çevrimdışı taraf hazır. librosa'da oktav hatasını doğrudan etkileyen iki parametre: `max_transition_rate` (varsayılan **35,92 oktav/saniye** — klarnet için çok gevşek) ve `switch_prob` (varsayılan **0,01**).

Çözüm 2 — Aday üretimi + dinamik programlama (RAPT, Praat)

pYIN'in Aşama 2'si aslında 1990'ların konuşma işleme literatüründen gelir. İki klasik:

RAPT (Talkin, 1995): Normalize edilmiş çapraz korelasyon (NCCF) ile her karede birkaç aday üretir, sonra dinamik programlama ile ötümlü/ötümsüz kararlarını ve perde yolunu *birlikte* çözer. Geçiş maliyetleri oktav sıçramalarını ve V/UV geçişlerini açıkça cezalandırır.

Praat (Boersma, 1993): Her kare için birden fazla otokorelasyon zirvesi bulur, sonra “perde sıçramalarından kaçınan ve yüksek korelasyonlu zirveleri tercih eden” bir yol seçer. Boersma'nın ek katkısı, filtrelenmiş sinyali **pencere fonksiyonunun normalize otokorelasyonuna bölmektir** — bu, pencerelemenin ACF zarfına yaptığı bozulmayı geri alır.

Bağımsız kanıt: Babacan ve ark., ICASSP 2013

Etiketli ve EGG ile hizalanmış geniş bir şarkı veritabanında altı yöntem karşılaştırıldı. Gross Pitch Error (GPE):

Yöntem	GPE (%)	Not
RAPT* (son-işlemeli)	0,65	en düşük kaba hata
RAPT	1,01	son-işleme öncesi
STRAIGHTv*	1,25	reverb'e en dayanıklı
PRAAT	1,41	ses sınırlarını en iyi bulan
YIN	—	en düşük Fine Pitch Error (en hassas), ama reverb'de en çok bozulan

İki sonucun altı çizilmelidir:

1. **Son-işleme tek başına** RAPT'ta %1,01 → %0,65 kazandırıyor; makale bu iyileşmenin özellikle RAPT ve YIN'de belirgin olduğunu söylüyor.
2. **YIN ince doğrulukta en iyi, kabalıkta değil.** Bu tam olarak bu projenin durumunun tarifidir: YIN'i atmayın, **kaba kararı başka bir katmana devredin**. YIN sente kadar doğru ölçer; hangi oktavda olduğuna karar vermek onun işi olmasın.

Gerçek zamanlı sorun ve çözümü

Tam Viterbi çevrimdışıdır (tüm parçayı görmesi gerekir). Canlı çalışma modunda **fixed-lag Viterbi** kullanılır: N kare geriden karar verirsiniz, sabit bir gecikme karşılığında geleceğe bakabilirsiniz.

Koddaki durum: core/include/klarivision/core/fixed_lag_tracker.hpp — lag_frames = 5, 512 örnek hop @ 48 kHz → ≈ 53 ms sabit karar gecikmesi, transition_width_cents = 700. 700 sent, bir oktavın (1200 sent) altındadır ama klarnetin on ikilisi olan 1902 sente göre çok dardır — yani mevcut ayar 3x hatalarını zaten büyük ölçüde cezalandırıyor. Ancak bu bileşen pitch_engine_v2 ile bağlı; **yin_v1** yolunda kullanılmıyor.

Çözüm 3 — Spektral harmonik kanıtlı veto ve puanlama

Bu aile, zaman-alanı algoritmasının verdiği adayı **frekans alanında sorguya çeker**: “Bu frekansta gerçekten enerji var mı, yoksa sadece otokorelasyon simetrisinin ürettiği bir hayalet mi?”

(a) SWIPE' — sadece asal harmonikler

Camacho & Harris, JASA 2008.

SWIPE, girdi spektrumuna en iyi uyan **testere dışı dalganın** temel frekansını arar. Karşılaştırma, sinyal spektrumu ile değiştirilmiş bir kosinüs arasındaki normalize iç çarpımla yapılır; pencere boyu, spektrumun ana loblarının genişliği kosinüsün pozitif loblarıyla eşleşecek şekilde seçilir.

SWIPE' varyantının kritik farkı: **yalnızca birinci ve asal harmonikleri** (1, 2, 3, 5, 7, 11...) kullanır. Neden işe yarar: f/2 gibi bir alt-harmonik adayın harmonik ızgarası f, 1,5f, 2f... noktalarına düşer; gerçek sinyalde 1,5f'de bir şey yoktur. Tüm harmonikleri sayan bir yöntem alt-harmonik yine de yüksek puanlar (çünkü çift indisli harmonikleri gerçek sinyallinkilerle çakıştır); asal harmoniklerle sınırlarsanız **alt-harmonik adayın kanıtsız kaldığı yerler açığa çıkar**.

Koddaki durum: core/include/klarivision/core/swipe_prime.hpp tam olarak bunu yapıyor — “birinci ve asal harmonikler pozitif destek alır, tüm tam sayı harmonik konumları normalizasyona katkı verir; bu alt-harmonik adayları açıklanabilir biçimde zayıflatır.” Doğru fikir, doğru uygulanmış — ama v2 isim alanında ve yin_v1 bunu kullanmıyor.

(b) İki yönlü uyumsuzluk — TWM

Maier & Beauchamp, JASA 1994.

Aday f_0 için hata iki yönlü hesaplanır:

- **Ölçülenden tahmin edilene:** her ölçülen spektral zirve, en yakın öngörülen harmoniğe olan uzaklığıyla cezalandırılır
- **Tahmin edilenden ölçülene:** her öngörülen harmonik, en yakın ölçülen zirveye olan uzaklığıyla cezalandırılır

İkinci yön oktav hatasını öldürendir. Gerçek f_0 'ın bir oktav altındaki bir aday, ölçülen tüm zirveleri gayet iyi “açıklar” (hepsi onun çift harmonikleridir) — birinci yön onu ayırt edemez. Ama o adayın **tek indisli harmonikleri karşılığında hiçbir zirve bulamaz**, ve ikinci yön bunu ağır cezalandırır.

Klarnet için kritik uyarı: TWM'yi olduğu gibi uygularsanız, klarnetin *gerçek* f_0 'ı da “eksik harmonikleri var” diye cezalandırılır (2f, 4f gerçekten yoktur). TWM'yi klarnete uyarlarken **öngörülen harmonik kümesini tek harmoniklerle (f, 3f, 5f, 7f) sınırlamalısınız**; aksi hâlde yöntem hatayı düzeltmek yerine üretir.

(c) Alt-harmonik toplama (SHS) ve harmonik çarpım spektrumu (HPS)

HPS: Spektrumu 2, 3, 4... katlarına sıkıştırıp çarparsınız; gerçek f_0 'da tüm harmonikler üst üste biner ve dev bir zirve oluşur. Basit, gerçek zamanlı çalışır. Zayıflığı: HPS çoğunlukla **bir oktav yukarı** kayar. Yaygın pratik düzeltme kuralı: seçilen perdenin **yarısında** bir zirve varsa ve genlik oranı bir eşiğin üstündeyse (5 harmonik için $\approx 0,2$), alt oktavı seç.

SHS (Hermes, 1988): Her spektral bileşenin bir alt-harmonik serisi ürettiğini varsayar ve bunları toplar — insan işitmesindeki *sanal perde* (virtual pitch) olgusuna karşılık gelir. Hermes, SHS'in **harmonik elek** (harmonic sieve) yöntemini benzer hesaplama maliyetiyle geride bıraktığını göstermiştir; sebep olarak SHS'in güncel perde-algı kuramlarıyla daha iyi örtüşmesini gösterir. Yöntem **telefon bandına yüksek geçiren süzölmüş konuşmada** (300 Hz altı kesik — yani temel frekansın fiziksel olarak yok olduğu durumda) da test edilmiştir; bu, “temeli olmayan sesin perdesi” problemi olarak klarnetin chalumeau bölgesiyle akrabadır.

(d) Kalıntı harmoniklerin toplamı — SRH

Drugman & Alwan, Interspeech 2011.

Otoregresif modelleme ve ters filtreleme ile **kalıntı (excitation) sinyali** çıkarılır; bu sinyal ses yolu rezonanslarından ve arka plan gürültüsünden büyük ölçüde arınmıştır. Perde bu kalıntının harmonikliği üzerinden ölçülür. 10 gürültülü deneyin 9'unda anlamlı iyileşme sağlamış, temiz koşullarda diğerleriyle eşdeğer kalmıştır. Optimum parametreler: **100 ms kare, 5 harmonik**. Klarnete doğrudan aktarılamaz (kaynak-filtre modeli konuşmaya özgüdür), ama fikir taşınabilir: *rezonans zarfını sinyalden çıkarıp saf harmonik yapıya bakmak*.

Koddaki durum: harmonic_arbitration.hpp içindeki spectral_existence() fonksiyonu bu ailenin minimal ve isabetli bir üyesi: aday frekansta gerçek enerji var mı, yoksa $2\times/3\times$ katı tarafından mı eziliyor. Eşik bilinçli olarak muhafazakâr tutulmuş (“çok katlı genlik baskınlığı”) ki normal biçimde yumuşak bir temel hayalet sanılmasın. Doğru mühendislik kararı.

Çözüm 4 — Sinir ağı tabanlı tahminçiler

Fikir: Periyodikliği elle tasarlanmış bir fonksiyonla aramak yerine, ağa doğrudan dalga formunu (veya CQT/spektrogramı) verip **perde kutularını sınıflandırmasını** öğretmek. Ağ, “hangi oktav” sorusunu tınıdan öğrenir — çünkü etiketli veride bir oktav yanlış cevap her zaman cezalandırılmıştır.

Model	Yıl	Özellik
CREPE	2018	Zaman-alanı dalga formu üzerinde derin CNN; yayımlandığında pYIN ve SWIPE'ı geçti. RWC-synth'te hata oranı taban çizgilerinden bir büyüklük mertebesi düşük
FCN-f0	2019	CREPE'i tam evrişimli hâle getirir (zero-padding yok, çıkışta lineer yerine evrişim katmanı, dropout yok, 8 kHz giriş). 30–1000 Hz aralığında 12,5 sent genişlikte 486 perde kutusu. Benzer doğrulukla belirgin şekilde daha az hesap → gerçek zaman için uygun
PENN	2023	Çapraz-alan (konuşma + müzik) perde ve periyodiklik tahmini
RMVPE	—	Özellikle vokal kayıtlarda en iyi (Vocadito %96,4; MIR-1K %96,0)
SwiftF0	2025	Sadece 95.842 parametre ; CPU'da CREPE'ten ~42x (bazı ölçümlerde 90x) hızlı

SwiftF0'ın ölçümleri: Temiz koşulda Cent Accuracy **%94,98**, Gross Error Accuracy **%97,04**. 10 dB SNR'de harmonik ortalama **%91,80** — CREPE'i **12 puandan fazla** geçiyor ve temizden gürültüye yalnızca 2,3 puan kaybediyor. Makale ayrıca şunu gözlemliyor: pYIN ve PENN ötümlü bölgelerde büyük boşluklar bırakıyor, CREPE ise sessiz bölümlerde bile perde raporlayıp yanlış tespit üretiyor.

Bağımsız kıyaslama (lars76/pitch-benchmark; 12 algoritma × 8 veri kümesi; harmonik-ortalama doğruluk):

Sıra	Model	Doğruluk
1	SwiftF0	%90,2
2	RMVPE	%87,2
3	CREPE	%85,3
4	PENN	%84,8
5	Praat	%84,7

Bu tablodaki en öğretici satır beşincisidir. Praat — 1993 tarihli klasik bir otokorelasyon + yol arama algoritması — saniyelik sesi **2,8 ms**'de işleyerek modern sinir ağlarının yalnızca 0,6 puan gerisinde kalıyor. Yani “sinir ağına geçmek” bu problemi çözmenin en verimli yolu değildir: **doğru kurulmuş klasik bir aday + yol mimarisi zaten oraya çok yakındır.**

Gerçekçi değerlendirme: iOS'ta Core ML ile SwiftF0/FCN-f0 boyutunda bir model çalışır. Ancak: (1) bu modellerin eğitim verisi ağırlıklı olarak *konuşma* ve *şarkıdır*, klarnetin tek-harmoniklik tınısı dağılım dışıdır; (2) bir ağın hatasını hata ayıklayamazsınız — bu projedeki gibi “neden bu kararı verdi” diyagnostiği (HAPTHypothesisDiagnostic , VPMSessionDiagnostic) kaybolur. Yine de bunu **turnuva içinde bir hakem/oy verici** olarak kullanmak çekici bir orta yoldur.

Çözüm 5 — Bayesçi harmonik model (model derecesi seçimiyle)

Shi, Nielsen, Jensen, Little, Christensen — IEEE/ACM TASLP 2019.

Fikir: Sinyali doğrudan harmonik model olarak yazın (f_0 ve L adet harmonik) ve **hem f_0 'ı hem harmonik sayısı L'yi birlikte çıkarım yapın**. f_0 , model derecesi ve voicing üzerine birinci mertebeden Markov süreçleriyle düzgünlük öncülleri konur.

Neden oktav hatasını yapısal olarak çözer: Bir oktav-altı aday, ancak **iki kat fazla harmonik** varsayarsanız veriyi açıklayabilir. Bayesçi çerçevede model derecesi arttıkça karmaşıklık cezası da artar (Occam usturası otomatik gelir). Makalenin verdiği somut örnek çarpıcıdır: hızlı NLS yöntemi bir karede perdede **72 Hz ve 6 harmonik** olarak tahmin ederken, Bayesçi yöntem aynı kareyi **143,2 Hz ve 3 harmonik** olarak çözer — yani tam olarak “iki katı frekans, yarısı harmonik” ikilemi doğru tarafa düşer.

Değerlendirme: Akademik olarak bu ailenin en zarif çözümü. Ama hesap yükü gerçek zamanlı bir iPad uygulaması için ağırdır ve klarnetin tek-harmoniklik yapısı için özel uyarılama gerektirir (genel L varsayımı yerine “1, 3, 5, 7” izgarası).

Çözüm 6 — Ucuz ama yüksek getirili mühendislik önlemleri

Bunlar teorik olarak “sıkıcı” ama YIN'in kendi tablosundaki %10 → %0,5 düşüşün büyük kısmını bunlar üretiyor. **Önce bunlar yapılmalıdır.**

(a) Arama aralığını fiziksel olarak daraltın

Enstrümanın çalamadığı hiçbir frekansı aramayın. Bir oktav-altı hata, ancak o oktav arama aralığındaysa mümkündür. Bb klarnetin en pes yazılı Mi'si duyulan $\approx 146,8$ Hz'dir; `minimum_frequency_hz = 110.0` bunun altında gereksiz alan bırakır.

Fakat aşırıya kaçmayın. `vpm_like.hpp` 'deki yorum ince tarafı doğru yakalamış: “estimator için 1500 Hz'lik gösterim aralığının üstünde küçük bir koruma bandı bırak — gerçek bir sınır notası, kendi f/2 periyodu kazanmadan önce değerlendirilmelidir.” Aralığı fazla daraltmak, sınırdaki notaları **sistematik olarak** yanlış oktava iter. Doğru kural: **gösterim aralığı dar, tahminci aralığı bir miktar geniş.**

(b) Pencere boyu

YIN'de $T_{\max}$ ile entegrasyon penceresi W **ayrıştırılmıştır** — bu, YIN'in spektral yöntemlere göre bir avantajdır. Kural: **entegrasyon penceresi beklenen en uzun periyottan kısa olmamalıdır**; aksi hâlde tahmin periyodun bazı fazlarında bozulur. Pratik denge **2–3 perde periyodudur**: uzun pencere pes notalarda kararlılık verir ama vibrato ve glissando'yu bulanıklaştırır. YIN makalesi ayrıca $T_{\max}$ 'ın “çok yüksek” ve “çok alçak” hatalar arasında bir **takas parametresi** olduğunu, toplam hatanın ara değerlerde minimum yaptığını belirtir — bu parametre deneysel olarak taranmalıdır.

Mevcut durumda `frame_length = 2048`, `minimum_frequency_hz = 110` → en uzun periyot ≈ 436 örnek (48 kHz'de), pencere 2048 → $\approx 4,7$ periyot. Makul, hatta biraz uzun.

(c) YIN Adım 6'yı ekleyin

Hataların üçte biri. Tek başına en yüksek getiri/maliyet oranına sahip değişiklik.

(d) Medyan filtre + yumuşatma

Literatürün standart reçetesi: **uzun mertebeli medyan filtre** (perde ikiye bölünmelerini ve katlanmalarını atar), ardından **küçük mertebeli yumuşatma filtresi** (kalan iz gürültüsünü temizler). Nedensel (causal) sistemlerde filtre uzunluğu doğrudan gecikmeye dönüşür; yaygın varsayılan **3 karedir**. Bu, kararlılık ile tepkisellik arasında bir takastır.

Ama Çözüm 1'deki YIN+S sonucunu unutmayın: **tek adaylı bir çıktıya uygulanan yumuşatma recall'ı düşürebilir**. Medyan filtre bir yamadır, çözüm değildir.

(e) Aşağı yönlü sıçrama için onay gecikmesi (histerezis)

Yerleşmiş bir konturdan aniden aşağı atlamayı hemen yayımlamayın, N kare onay bekleyin. Yukarı atlamalar için aynı kısıtı koymayın — çünkü alt-harmonik hataları **aşağı** yöndedir.

Koddaki durum: Bu zaten var ve doğru asimetriyle kurulmuş: `VPMLikeTracker(downward_harmonic_confirmations = 3)`, HAPT'ta `downward_harmonic_confirmations = 2 + downward_harmonic_jump_ratio_cents_tolerance = 30`, ve `analysis_engine.cpp` 'de `pending_yin_jump / pending_yin_confirmations`. `vpm_like.hpp` 'deki yorum sınırı da doğru çiziyor: “sadece yerleşmiş bir konturdan aşağı yönlü harmonik sıçramayı geciktirir, sıradan perde hareketini asla yumuşatmaz.”

(f) İki eşikli histerezis (onset / sustain)

Yeni bir nota için yüksek periyodiklik eşiği, halihazırdaki kontura yakın bir tahmin için düşük eşik. `hapt.hpp` 'de mevcut: `onset_periodicity_threshold = 0.55`, `sustain_periodicity_threshold = 0.32`, `sustain_max_cents_from_contour = 180`.

(g) Ön-filtreleme

Bant sınırlaması, DC ve düşük frekans gürültüsünün sahte uzun periyot kanıtı üretmesini engeller. Genel kural: **50 Hz altında perde aranmamalıdır**.

(h) MPM'in “clarity” parametresi

McLeod Pitch Method (2005), normalize kare fark fonksiyonu (NSDF) kullanır ve zirve seçiminde bir `k` sabitiyle eşik uygular: en yüksek maksimumun `k` katı üstündeki ilk anahtar maksimum seçilir. McLeod'un kendi ifadesiyle bu sabit “güçlü harmoniklerden kaynaklanan zirveleri seçmeyecek kadar büyük, ama istenmeyen vuru veya alt-harmonik sinyalleri seçmeyecek kadar küçük” olmalıdır. MPM alçak geçiren filtre kullanmaz, bu yüzden keman gibi yüksek harmonik içerikli seslerde de çalışabilir. Projede `core/src/mpm.cpp` olarak mevcuttur.

Çözüm 7 — Diğer literatür yaklaşımları (tamlik için)

- **Fourier serisi yaklaşımı** (IETE Technical Review, 2018): Yarı-periyodik sinyali tam periyodik bir sinyalle yaklaşıklar ve en iyi

yaklaşımı veren f_0 'ı seçer; harmoniklerin göreceli güçlerini analiz eder. Hem güçlü harmonikli hem de neredeyse sinüzoidal içerikte etkili olduğu bildiriliyor.

- **“Fourier'in Fourier'i”:** Spektrumun kendisinin periyodikliğini ölçmek (kepstruma akraba), ardından parabolik interpolasyonla çözünürlük artırmak.
- **Zaman pencereleri arası harmonik genlik uydurma:** Ardışık pencerelerde karşılık gelen harmoniklerin genliklerini uydurarak oktav hatasından kaynaklanan *sahte harmonikler* teşhis etmek.
- **Oylama / topluluk yöntemleri:** Birden fazla tahmincinin zamansal ve frekanssal hizalamayla oylanması. Bu projenin dört-motorlu “turnuva” mimarisi zaten bunun altyapısıdır.

5. Tuzaklar — yapılmaması gerekenler

1. **Sadece son-işleme ile çözmeye çalışmak.** pYIN makalesinin YIN+S kontrolü bunun recall'ı düşürdüğünü gösteriyor. Çoklu aday olmadan yumuşatma tavanlıdır.
2. **Genel “2x kontrol et” reçetelerini klarnete uygulamak.** Klarnette 2f'nin zayıf/yok olması *normaldir*, hata kanıtı değildir. Yanlış kurulmuş bir veto sistemi doğru notaları reddetmeye başlar.
3. **Arama aralığını aşırı daraltmak.** Sınır notalarını sistematik olarak yanlış oktava iter.
4. **Eşiği (threshold) oynayarak çözmeye çalışmak.** YIN makalesi açıkça söylüyor: bu eşiğin kesin değeri hata oranlarını kritik biçimde etkilemez. pYIN'in tüm varlık sebebi zaten “doğru eşik diye bir şey yok, hepsini birden dene”dir.
5. **Motor izlerini bayatlamış çıktılarıyla karşılaştırmak.** outputs/ altındaki izler farklı derlemelerden kalmış olabilir; kıyastan önce tazeleyin.

6. KlariVision için somut uygulama sırası

Kodun durumu şu şekilde özetlenebilir: **parçalar var, kablolar bağlı değil.**

Bileşen	Durum
swipe_prime.hpp (asal harmonik desteği)	Var — v2 'de; yin_v1 kullanmıyor
harmonic_arbitration.hpp (hayalet alt-harmonik vetosu)	Var
fixed_lag_tracker.hpp (nedensel Viterbi, ~53 ms)	Var — v2 'de; yin_v1 kullanmıyor
yin_candidates() (çoklu aday üretimi, C++)	Var
pyin.py (librosa pYIN)	Var — çevrimdışı
Aşağı sıçrama onay gecikmesi	Var — doğru asimetriyle
yin.py 'de YIN Adım 6	Yok
yin_v1 'in çoklu aday Viterbi'ye bağlaması	Yok

Önerilen sıra:

1. **yin.py 'ye Adım 6'yı ekleyin** ve minimum_frequency_hz 'i enstrümanın gerçek alt sınırına çekin. En düşük maliyetli, ölçülebilir kazanç (YIN'in kendi ölçümünde kalan hataların üçte biri).
2. **Çevrimdışı referans olarak pYIN'i benchmark'a alın** ve max_transition_rate 'i klarnet için daraltın (varsayılan 35,92 oktav/sn çok gevşek). Bu size “ulaşılabilir en iyi” tavanını verir; sonraki her değişikliği bu tavana karşı ölçersiniz.
3. **yin_v1 'i tek-cevaptan çoklu-adaya çevirin.** yin_candidates() zaten üretiyor; FixedLagPitchTracker 'ı yin_v1 yoluna da bağlayın. Bu, pYIN Aşama 2'nin nedensel karşılığıdır ve canlı modda ~53 ms gecikmeyle çalışır.
4. **Emisyon skorunu swipe_prime + spectral_existence ile besleyin** — ama harmonik ızgarasını klarnet için **tek harmoniklere (1, 3, 5, 7)** göre kurun, standart tam-sayı ızgarasına göre değil. Aday puanı = (YIN periyodikliği) × (asal/tek harmonik spektral desteği).
5. Bunlar bittikten sonra hâlâ boşluk varsa, **SwiftF0 / FCN-f0 boyutunda bir modeli turnuvaya beşinci hakem olarak** eklemeyi değerlendirin.

En kritik sonuç: YIN'i değiştirmeniz gerekmiyor. Babacan ve arkadaşlarının bulgusu net — YIN ince perde doğruluğunda (FPE) en iyi, kaba hatalarda (GPE) değil. YIN'in *ölçüm* rolünü koruyun, *oktav kararı* rolünü ondan alın ve aday-üretimi + yol-seçimi + harmonik-kanıt üçlüsüne devredin. pYIN'in tüm hikâyesi tam olarak budur.

7. Kaynakça

1. de Cheveigné, A. & Kawahara, H. — *YIN, a fundamental frequency estimator for speech and music*, JASA 111(4):1917–1930, 2002.
recherche.ircam.fr/equipes/pcm/cheveign/ps/2002_JASA_YIN_proof.pdf
2. Mauch, M. & Dixon, S. — *PYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions*, ICASSP 2014.
web.space.eecs.qmul.ac.uk/s.e.dixon/pub/2014/MauchDixon-PYIN-ICASSP2014.pdf
3. Babacan, O., Drugman, T., d'Alessandro, N., Henrich, N. & Dutoit, T. — *A Comparative Study of Pitch Extraction Algorithms on a Large Variety of Singing Sounds*, ICASSP 2013.
arxiv.org/pdf/1912.12609
4. Camacho, A. & Harris, J. G. — *A sawtooth waveform inspired pitch estimator for speech and music (SWIPE')*, JASA 124(3):1638, 2008.
audiolabs-erlangen.de/.../2008_CamachoH_SWIPE_JASA.pdf
5. Maher, R. C. & Beauchamp, J. W. — *Fundamental frequency estimation of musical signals using a two-way mismatch procedure*, JASA 95(4):2254, 1994.
6. Hermes, D. J. — *Measurement of pitch by subharmonic summation*, JASA, 1988.
pubmed.ncbi.nlm.nih.gov/3343445
7. Drugman, T. & Alwan, A. — *Joint Robust Voicing Detection and Pitch Estimation Based on Residual Harmonics (SRH)*, Interspeech 2011.
isca-archive.org/interspeech_2011/drugman11_interspeech.pdf
8. Talkin, D. — *A Robust Algorithm for Pitch Tracking (RAPT)*, Speech Coding and Synthesis, 1995.
9. Boersma, P. — *Praat: pitch analysis by filtered autocorrelation*, 1993 / kılavuz.
fon.hum.uva.nl/praat/manual/pitch_analysis_by_filtered_autocorrelation.html
10. McLeod, P. & Wyvill, G. — *A Smarter Way To Find Pitch (MPM)*, ICMC 2005.
quod.lib.umich.edu/i/icmc/bbp2372.2005.107
11. Shi, L., Nielsen, J. K., Jensen, J. R., Little, M. A. & Christensen, M. G. — *Robust Bayesian Pitch Tracking Based on the Harmonic Model*, IEEE/ACM TASLP, 2019.
arxiv.org/pdf/1905.08557
12. Kim, J. W., Salamon, J., Li, P. & Bello, J. P. — *CREPE: A Convolutional Representation for Pitch Estimation*, ICASSP 2018.
arxiv.org/abs/1802.06182
13. Ardaillon, L. & Roebel, A. — *Fully-Convolutional Network for Pitch Estimation of Speech Signals (FCN-f0)*, Interspeech 2019.
isca-archive.org/interspeech_2019/ardaillon19_interspeech.pdf
14. Nieradzki, L. — *SwiftF0: Fast and Accurate Monophonic Pitch Detection*, 2025.
arxiv.org/pdf/2508.18440
15. lars76 — *pitch-benchmark: 12 algoritma x 8 veri kümesi karşılaştırması*.
github.com/lars76/pitch-benchmark
16. *Octave Error Reduction in Pitch Detection Algorithms Using Fourier Series Approximation Method*, IETE Technical Review 36(3), 2018.
tandfonline.com/doi/full/10.1080/02564602.2018.1465859
17. *High accuracy and octave error immune pitch detection algorithms*, Archives of Acoustics.
sound.eti.pg.gda.pl/papers/pitch_detection_algorithms.pdf
18. *Harmonic Product Spectrum* — UCSD Music.
musicweb.ucsd.edu/~trsmth/analysis/Harmonic_Product_Spectrum.html
19. *librosa.pyin* dokümantasyonu (`max_transition_rate`, `switch_prob`).
librosa.org/doc/main/generated/librosa.pyin.html
20. von dem Kneesebeck, A. & Zölzer, U. — *Comparison of Pitch Trackers for Real-Time Guitar Effects*, DAFx-10.
dafx10.iem.at/papers/VonDemKneesebeckZoelzer_DAFx10_P102.pdf

Bu rapor, birincil kaynakların tam metinlerinden (YIN 2002 ve pYIN 2014 makaleleri dahil) çıkarılan sayısal sonuçlarla ve KlariVision kod tabanının doğrudan incelenmesiyle hazırlanmıştır. Koda ilişkin tespitler `src/klarivision/pitch/yin.py`, `src/klarivision/pitch/pyin.py`, `core/src/analysis_engine.cpp`, `core/include/klarivision/core/hapt.hpp`, `harmonic_arbitration.hpp`, `swipe_prime.hpp`, `vpm_like.hpp` ve `fixed_lag_tracker.hpp` dosyalarına dayanmaktadır.